PRISM: the parameter-inference workflow

Simulation-based inference for the hair-bundle model, from the prior to the posterior

13 October 2026 • checked against the repository at commit `cb2b0de`

Scope of this summary

This document describes the *simulation-based parameter-inference* line of PRISM only: how the prior is built, how simulations are generated, what the network is given and what it returns, how it is trained, checked and used, and how a round can be narrowed around one observation. It is written from the code and the reader guides in the repository (file paths are given so each statement can be traced). The effective-temperature (fluctuation–dissipation) line, the Nadrowski-to-Hopf reduction map and the graphical front end are outside its scope. Numbers are the code's defaults unless a run says otherwise; the planned retrain differs in a few of them, and each such difference is flagged where it occurs.

Contents

1 The workflow at a glance **2**
1.1 The three observation modes . . . 3

2 Inputs, outputs and coordinates **4**
2.1 What goes in . . . 4
2.2 The model and its parameters . . . 4
2.3 What comes out . . . 4
2.4 Coordinates: from the box to the latent space . . . 5

3 Before training: the one-off setup **5**
3.1 Recording geometry and the grids . . . 5
3.2 The stability-screened prior . . . 6
3.3 The tier-1 constraint (temperature in place of the force scale) . . . 6
3.4 The Fisher rotation . . . 6
3.5 Guards and the identity of a run . . . 7

4 Generating the simulations **7**
4.1 The solver . . . 7
4.2 The geometry of a batch . . . 7
4.3 One training batch, step by step . . . 7
4.4 The susceptibility (χ) observation . . . 8
4.5 The simulation cache, memory and resumption . . . 9

5 What the network sees: the conditioning row **9**
5.1 The 41 summary statistics . . . 9
5.2 Flags, duration and the mode-specific block . . . 10
5.3 Pre-processing before the network . . . 10

6 The network **11**
6.1 Architecture . . . 11

6.2 The χ set encoder . . . 11
6.3 The output: a flow on the latent coordinate . . . 12
6.4 Training . . . 12
6.5 Why these choices (briefly) . . . 12

7 Calibration, informativeness and predictive checks 12
7.1 The calibration set . . . 12
7.2 Rank uniformity (SBC) . . . 13
7.3 Joint coverage (TARP) . . . 13
7.4 Informativeness . . . 13
7.5 The verdict, and how to read it . . . 14
7.6 Diagnostics that ask whether the information is there . . . 14
7.7 What the August 2026 retrain showed . . . 14

8 Running the posterior on an observation 14
8.1 What an observation is . . . 14
8.2 Simulated and measured observations . . . 15
8.3 Checks before sampling . . . 16
8.4 Sampling and outputs . . . 16

9 Narrowing rounds (TSNPE) 16
9.1 Draw from the truncated prior, never from the posterior . . . 17
9.2 How a round runs . . . 17
9.3 The safety rules . . . 17
9.4 Reading a round . . . 17

10 Artifacts, reproducibility and operation 18
10.1 The artifact store . . . 18
10.2 Reproducibility . . . 19
10.3 Hardware, memory and long runs . . . 19
10.4 Front ends . . . 19

11 What the data can say, and what is still open 20
11.1 Identifiability . . . 20
11.2 Approximations and leads in the pipeline itself . . . 21
11.3 Open questions . . . 21

A Defaults by stage 22

B Where things live 23

1 The workflow at a glance

PRISM recovers the parameters of a stochastic model of a hair bundle from a record of the bundle's tip displacement. The model has no tractable likelihood, so the program uses *simulation-based inference*, in its

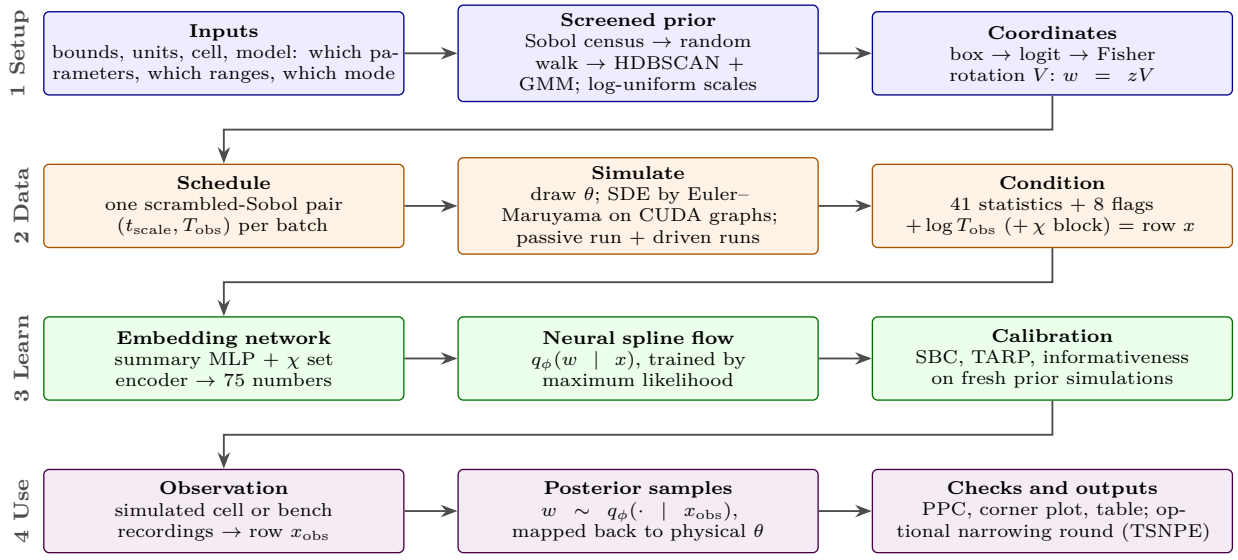


Figure 1: The parameter-inference workflow. Blue: setup, once per posterior; orange: training-data generation, one batch at a time; green: learning and calibration; violet: use on an observation.

amortised neural-posterior-estimation form (NPE, run through the `sbi` package's `SNPE` with a neural spline flow [1, 2, 3, 4]). The abstract view is:

1. **Fix the problem.** A bounds file lists which parameters are inferred and their ranges; that choice also fixes the *observation mode* (what a recording must contain).
2. **Build a prior** over the model's non-dimensional parameters by screening the box for dynamics that stay finite, and give the three physical scales their own prior.
3. **Choose coordinates**: map the box to an unbounded latent space and rotate it into the eigenbasis of a simulation-based Fisher matrix.
4. **Simulate.** For each of thousands of batches, draw one recording geometry (a time scale and a recording length), draw parameters from the prior, integrate the stochastic differential equations, and put the traces on the recording's clock.
5. **Condition.** Reduce each simulated recording to a fixed-width vector x: 41 summary statistics, 8 validity flags, $\log T_{\text{obs}}$, and (in susceptibility mode) a masked set of lock-in measurements.
6. **Train** an embedding network and a conditional neural spline flow $q_\phi(w \mid x)$ on the pairs (w, x), where w is the latent, rotated coordinate of the parameters.
7. **Calibrate** on fresh prior simulations (simulation-based calibration, TARP, informativeness).
8. **Infer** on an observation (a simulated cell or bench recordings): sample the posterior, map the samples back to physical units, and check them by re-simulating (posterior-predictive check).
9. **Optionally narrow**: truncate the *prior* to a high-density region around one observation and repeat steps 4–8 on that region (truncated sequential NPE).

Three design decisions that shape everything below

- **The network sees summaries, not traces.** Each recording is reduced to hand-built statistics (Section 5); the flow never sees the raw time series.
- **One network for all recording geometries.** Every training batch has its own time scale t_{scale} and recording length T_{obs}, and $\log T_{\text{obs}}$ is an input, so a single posterior handles recordings of 1–60 s and the whole t_{scale} range.
- **The mode is not a switch.** The bounds file declares what is inferred, and therefore what must be measured: a passive recording, a passive recording plus one driven recording, or a passive recording plus several single-tone driven recordings.

1.1 The three observation modes

The bounds file fixes the mode, and the mode fixes what a recording must contain and how wide the conditioning row is (Table 1). A posterior is trained in one mode and loads only in that mode.

Mode	What the observation contains	Conditioning width	Inferred d
Spontaneous	one unforced (passive) recording	50	12
Forced	the passive recording, plus one driven recording at a known amplitude, frequency and phase; the four forcing numbers are appended to x	54	13
Susceptibility (χ)	the passive recording, plus K single-tone driven recordings (up to 12; six by default for a simulated observation); a lock-in turns each into a complex response	122	13

Table 1: Observation modes. Widths are for the master bounds files (`Resources/Bounds/nadrowski/`): 50 is the 41 features, the 8 validity flags and $\log T_{\mathrm{obs}}$; 54 adds the four forcing numbers; 122 adds the $12 \times 6 = 72$-wide χ block. The spontaneous box has no force scale, so $d = 12$.

2 Inputs, outputs and coordinates

2.1 What goes in

- **Bounds** (`Resources/Bounds/<model>/*.txt`): the parameters to infer, their order and their ranges, plus the forcing block. The order of the file is the order of the output vector.
- **Units** (`Resources/Units/<model>/units.txt`): what the numbers mean; for the Nadrowski cells, `nm ms pN kHz`.
- **Cell** (`Resources/Cells/<model>/*.txt`): one named parameter set, used as the ground truth for simulated inference and as the source of the dimensional constants. A cell's model is the name of its parent folder.
- **A model**: built in (Nadrowski, Hopf, BP) or user-defined (`Resources/Models/`). The Nadrowski model is the one described here.
- **An observation**: either simulated from a cell, or bench recordings (a passive trace and, in χ mode, single-tone recordings given as a path and a drive frequency each, plus the physical drive amplitude F_0 and the recording length).

2.2 The model and its parameters

The Nadrowski model has three state variables: the bundle displacement x (the observable), the adaptation-motor position x_a, and a slow variable c that feeds back on the motor. In non-dimensional (ND) form, with open probability p and gating-spring extension x_{gs},

$$p = \left[1 + a\, e^{-\beta(x - x_a)}\right]^{-1}, \qquad a = e^{\Delta E + \beta/2}, \qquad x_{gs} = x - x_a - p, \tag{1}$$

$$\mathrm{d}x = \left[-(x_{gs} + k\,x) + F(t)\right]\mathrm{d}t \;+\; \sqrt{2/(N\beta)}\;\mathrm{d}W_x, \tag{2}$$

$$\mathrm{d}x_a = \tfrac{1}{\lambda}\left[x_{gs} - f_{\max}(1 - s\,c)\right]\mathrm{d}t \;+\; \sqrt{2\,\tilde{T}/(N\beta\lambda)}\;\mathrm{d}W_a, \tag{3}$$

$$\mathrm{d}c = \tfrac{1}{\tau}\,(p - c)\,\mathrm{d}t \;+\; \tfrac{1}{\tau}\sqrt{2\,\tau_c\,p(1-p)/N}\;\mathrm{d}W_c. \tag{4}$$

The noise is diagonal and independent. The physical trace is $x_{\mathrm{phys}} = x_{\mathrm{scale}}\,x$ and physical time is $t = t_{\mathrm{scale}}\,t_{\mathrm{ND}}$; the drive $F(t)$ has physical amplitude F_0 and ND amplitude F_0/f_{scale} (`core/Models/nadrowski_model.py`, `core/forcing.py`).

Three of the ten ND parameters (N, $\tilde{T}$, τ_c) enter *only* the noise amplitudes. They are seen through the statistics of the fluctuations alone, which is why they are the hardest to recover (Section 11).

2.3 What comes out

- **From the network:** a conditional density $q_\phi(w \mid x)$ over the latent coordinate of the d inferred parameters (12 or 13). In practice, samples (1000 per call by default) and their log density.
- **From an inference run:** samples in physical units; the 5/50/95 % summary per parameter; a corner plot; a posterior-predictive check (every sample re-simulated at the observation's geometry); overlay figures (spectrum band, cycle average, best-fit summary-statistics table); the fraction of observed statistics inside the predictive 90 % band; and a record in the artifact store.

Name	Symbol	Master range	Where it acts in the model
`k`	k	(0.01, 5)	drift of x: restoring term $-k\,x$
`lam`	λ	(0.1, 50)	drift of x_a (divided by λ) and noise of x_a ($\propto 1/\sqrt{\lambda}$)
`f_max`	$f_{\max}$	(0.05, 30)	drift of x_a: motor force $f_{\max}(1 - sc)$
`tau`	τ	(0.0001, 2)	relaxation time of c; also scales the noise of c
`tau_c`	τ_c	(0.005, 5)	noise of c only
`s`	s	(0, 3)	drift of x_a: strength of the c feedback on the motor force
`delta_E`	ΔE	(0, 30)	open probability: offset in $a = e^{\Delta E + \beta/2}$
`beta`	β	(0.5, 100)	open-probability steepness; noise of x and x_a
`n`	N	(10, 300)	noise only: all three amplitudes scale as $1/\sqrt{N}$
`temp`	$\tilde{T}$	(0.05, 10)	noise of x_a only
`x_scale`	x_{scale}	(10, 100) nm	length scale of the trace
`t_scale`	t_{scale}	(1, 40) ms	time scale: $t = t_{\text{scale}} t_{\text{ND}}$
`f_scale`	f_{scale}	(1, 1000) pN	force scale (drive amplitude); absent in the spontaneous box. In the tier-1 box it is replaced by the bath temperature $T \in (280, 310)$ K (Section 3.3)

Table 2: The ten ND parameters and three scales of `Resources/Bounds/nadrowski/master.txt`. The forcing block (`amp`, `freq`, `phase`, `offset`) is declared in the same file but is drawn from its own prior in forced mode and is never inferred.

- **From the checks:** a calibration verdict (SBC and TARP), the informativeness of the posterior, and optionally identifiability diagnostics.

2.4 Coordinates: from the box to the latent space

The flow does not work on the physical parameters θ but on a latent coordinate w obtained by a chain of fixed bijections:

$$\theta \in \prod_{i=1}^{d} [\mathrm{lo}_i, \mathrm{hi}_i] \xrightarrow{\text{affine}} s \in (0,1)^d \xrightarrow{\text{logit}} z \in \mathbb{R}^d \xrightarrow{\text{rotation}} w = zV \in \mathbb{R}^d, \tag{5}$$

with $s_i = (\theta_i - \mathrm{lo}_i)/(\mathrm{hi}_i - \mathrm{lo}_i)$ and $z_i = \mathrm{logit}(s_i) = \ln\big(s_i/(1-s_i)\big)$, where V is the orthogonal Fisher rotation of Section 3.4. Sampling runs the chain backwards: $w \sim q_\phi(\cdot \mid x)$, then $z = wV^\top$, then $\theta_i = \mathrm{lo}_i + \sigma(z_i)(\mathrm{hi}_i - \mathrm{lo}_i)$. A sample can therefore never leave the box, and the density in θ-space is $q_\phi(w \mid x)$ times the Jacobian of the chain, which does not depend on ϕ (`core/SBI/reparam.py`, class `TransformedPosterior`). All coordinates are linear in the box: the option to place a parameter on a log scale exists, but the list of log parameters is empty.

3 Before training: the one-off setup

3.1 Recording geometry and the grids

Every simulation runs on one *fine* non-dimensional grid, and each batch uses a prefix of it. The constants come from the configuration and the bounds (`core/config.py`, `core/sim_config.py`):

Quantity	Value	Meaning
frame interval Δt_{exp}	1 ms	the 1000-frames-per-second recording clock
recording length T_{obs}	1–60 s	range the posterior is trained for
finest ND step $\Delta t_{\min}$	$\Delta t_{\text{exp}}/t^{\text{hi}}_{\text{scale}} = 0.025$	fine integration step: frame interval over the largest time scale
transient discarded	100 ND units = 4000 steps	lets the state forget its initial condition
fine grid	$\approx 2.4 \times 10^6$ points	covers 60 s/$t^{\text{lo}}_{\text{scale}}$ ND units
segment length	100 000 steps	integration is split into segments (memory cap)
steps per batch, ceiling	300 000	batches that would need more are not generated

Table 3: Geometry constants for the master Nadrowski bounds.

3.2 The stability-screened prior

The ND parameters live in a ten-dimensional box, and not every point of it gives a bounded trajectory. The prior is built so that it puts its mass on parameters whose simulation stays finite (`build_prior`, `core/orchestrator.py`; `core/SBI/Priors/prior.py`).

1. **Census.** A scrambled Sobol sequence covers the box uniformly on its linear scale. Every candidate is integrated for 1000 ND time units and is *stable* if its whole trajectory stays finite. By default 50 rounds of one device batch each are screened. Only the ND block is screened; the three scales do not change the ND dynamics.
2. **Random-walk densification.** Parents are drawn with replacement from all accepted points, moved by one Gaussian step of standard deviation 0.01 (in each parameter's own units), and kept if they are inside the box and finite. The walk stops at 175 000 accepted sets or when a round accepts nothing.
3. **Mixture fit.** The accepted points are mapped to the latent coordinate z. HDBSCAN [8] (minimum cluster size 50, minimum samples 10) is run only to choose the *number* of components; a full-covariance Gaussian mixture with that many components is then fitted to *all* accepted points. The prior is this mixture pushed back through the box, so its support is exactly the bounds file's box. On the master boxes nearly the whole box is stable and the result is close to uniform.

The **scale block** has its own prior: log-uniform for every parameter whose name contains "scale" (and uniform for T in the tier-1 box). The training prior is the product of the ND mixture and the scale prior. In forced mode a separate **forcing prior** supplies the drive parameters; training draws from it and never infers it. The finished prior is stored as an artifact and named by the mixture's fingerprint, which the training cache uses as part of its identity.

3.3 The tier-1 constraint (temperature in place of the force scale)

On the master box the gating-spring energy is sampled twice, once as $N\beta$ in the ND block and once as $f_{\mathrm{scale}}\, x_{\mathrm{scale}}$ in the scale block, and nothing makes them agree. Their ratio is an implied bath temperature, and over 1.13 million training rows it spanned 14–6292 K (5th to 95th percentile), with only 2.27 % of rows in 280–310 K. The tier-1 box (`Resources/Bounds/nadrowski/master_tier1.txt`) therefore declares the bath temperature $T \in (280, 310)$ K in the column of f_{scale}, and the force scale is *derived* wherever θ is split for simulation (`core/SBI/derived.py`):

$$f_{\mathrm{scale}} \;=\; \frac{N\,\beta\,k_B\,T}{x_{\mathrm{scale}}}, \qquad k_B = 1.380649 \times 10^{-2}\ \mathrm{pN\,nm\,K^{-1}}. \tag{6}$$

β is never the derived quantity, because it is a drift parameter and the stability screen works on the ND block alone. The training target keeps T (the latent is taken before the derivation), the simulator runs at the derived f_{scale}, and the prior stays a proper 13-dimensional density. In χ mode T rescales every $|\chi|$ by one common factor, so it is only weakly informed and is close to degenerate with N.

3.4 The Fisher rotation

Some parameter combinations are nearly degenerate (for example k with x_{scale}), which makes the latent posterior a thin oblique ridge that is hard for a flow to fit. A rotation of the latent space aligns the axes with the directions the data can and cannot constrain (`core/SBI/decorrelate.py`, `build_latent_fisher_rotation`):

1. Choose 8 operating points (the ground truth, or prior draws).
2. At each, perturb each latent coordinate by ± 0.1 (central differences) and simulate an ensemble of 48 recordings for each perturbation; the conditioning features are computed for each.
3. Standardise by the locally measured feature noise (floored at 10^{-9}) to get the Jacobian J of the features with respect to z, form $F = J^{\top} J$, and average F over the operating points.
4. V is the matrix of eigenvectors of F; the eigenvalues are recorded. Directions the data constrain well come first.

In χ mode the Jacobian uses $41 + 3K$ features (for each probe: $\log|\chi|$, cos, sin; the probe's frequency, mask and cycle count are left out because they are either independent of θ or would amplify noise), at a cost of $K + 1$ simulations per evaluation. The rotation needs no trained posterior and is on in every mode. It is orthogonal, so it adds and removes no information; it only changes how easy the density is to fit. A fresh run computes V;

a resumed run, or a refit of a finished cache, reuses the one stored with the cache; a narrowing round reuses its parent's. The operating points are drawn from an unseeded random stream, so two fresh runs give different rotations, and a stored V must travel with the data it was used for.

3.5 Guards and the identity of a run

Before any compute is spent, the stages check a list of conditions and refuse with a named field when one fails (`core/refusals.py`, `run_guards`): that the chi configuration was chosen deliberately, that the prior and posterior fingerprints match, that a user model is in sync with its stored copy, and that the observation mode matches the bounds. The cost of the run is printed as a `[budget]` line. The simulation cache is identified by the number of batches, the batch width, the prior's fingerprint, the chi constants and the rotation flag; it is deliberately *not* keyed on the flow's hyper-parameters, so a finished cache can be refitted at another flow width without simulating again.

4 Generating the simulations

4.1 The solver

Each trajectory is a path of the stochastic differential equations of Section 2.2, integrated by the Itô Euler–Maruyama scheme [9] with diagonal noise,

$$X_{n+1} = X_n + f(X_n, F_n)\,\Delta t + g(X_n) \odot \xi_n \sqrt{\Delta t}, \qquad \xi_n \sim \mathcal{N}(0, I), \tag{7}$$

(`core/Solvers/sdeint.py`, `core/Simulator/simulator.py`). The default solver, `euler_compiled`, is a TorchScript step replayed from CUDA graphs (50 steps per captured graph). On the development card it ran 123 349 steps per second against 12 639 for the eager solver, and a two-arm check with 3072 rows per arm found the peak-frequency distributions indistinguishable (Kolmogorov–Smirnov $D = 0.002$). The cost of a batch is set by the number of *sequential* steps, so making a batch wider is nearly free while making a recording longer is not. The grid is cut into segments of 100 000 steps, each starting from the previous segment's last state. The built-in models start their position-like variables at random integers in $[0, 9]$ and the probability-like variable at 0; the 100-unit transient is discarded, so the start hardly matters.

4.2 The geometry of a batch

All rows of a batch share one recording geometry, because they are integrated as one tensor on one grid. The geometry of batch k is a pair $(t_{\mathrm{scale}}^{(k)}, T_{\mathrm{obs}}^{(k)})$ from a precomputed *schedule* (`_batch_schedule`, `core/SBI/pipeline.py`):

- a scrambled two-dimensional Sobol sequence, mapped to be log-uniform in t_{scale} over the bounds (1–40 ms) and log-uniform in T_{obs} over 1–60 s;
- drawn three times over and filtered: a pair is dropped if the batch would need more than 300 000 fine steps, so the corner of long recordings at small t_{scale} is absent from training (on the master box a 60 s recording needs $t_{\mathrm{scale}} \gtrsim 9$ ms; a 7 s one fits everywhere);
- stored in the simulation cache's header, so a resumed run reuses it instead of redrawing.

With $t_{\mathrm{scale}}^{\mathrm{hi}}$ the top of the bounds, the batch's geometry on the fine grid is

$$m_k = \max\bigl(1, \mathrm{round}(t_{\mathrm{scale}}^{\mathrm{hi}} / t_{\mathrm{scale}}^{(k)})\bigr), \qquad N_k = \lfloor T_{\mathrm{obs}}^{(k)} / \Delta t_{\mathrm{exp}} \rfloor, \qquad n_k = 4000 + N_k\, m_k, \tag{8}$$

where m_k is the integer stride that puts the fine grid on the recording clock (to within the rounding discussed in Section 11), N_k the number of frames and n_k the number of fine steps to integrate (transient included). Diversity over geometry comes from the number of batches (5000 by default); diversity over parameters comes from the rows inside a batch.

4.3 One training batch, step by step

For each batch k (`gen_training_data`, `core/SBI/pipeline.py`), with R rows (the device batch; 2048 in the planned retrain):

1. **Geometry.** Read $(t_{\text{scale}}^{(k)}, T_{\text{obs}}^{(k)})$ and compute m_k, N_k, n_k from Eq. (8).
2. **Parameters.** Draw R latent vectors from the training prior, lift them to physical θ through the inverse of the chain in Section 2.4, and split θ into the ND block and the scale block.
3. **Override the time scale.** Overwrite every row's t_{scale} with $t_{\text{scale}}^{(k)}$ (in physical space) and recompute the latent vector from the overwritten θ. *That* recomputed vector is the training target, so the target always agrees with what was simulated.
4. **Tier-1 derivation** (if the box declares T): compute $f_{\text{scale}} = N\beta k_B T / x_{\text{scale}}$ for the simulator only; the target keeps T.
5. **Simulate** the passive (spontaneous) trajectories on the first n_k fine steps, drop the first 4000, keep every m_k-th remaining point, and redimensionalise: $x = x_{\text{scale}}\, x_{\text{ND}}$ (plus an offset where a model has one). The resulting frames are treated as $\Delta t_{\text{exp}} = 1$ ms apart.
6. **Mode-specific runs.**
   - *Spontaneous:* nothing further.
   - *Forced:* draw the four drive numbers (`amp`, `freq`, `phase`, `offset`) per row from the forcing prior; simulate a second, driven trajectory; the forced-response group of statistics (G) is computed from it, and the four numbers are appended to the row.
   - *Susceptibility (χ):* simulate K further driven trajectories, one per probe, and lock each in (Section 4.4).
7. **Summarise.** Compute the 41 statistics and 8 flags of each passive trajectory (in sub-batches of 256 rows on the GPU), append $\log T_{\text{obs}}^{(k)}$ and the mode-specific block, and store the row and its target. Trajectories that are non-finite, exactly constant or larger than 10^{15} in magnitude are counted and reported.
8. **Commit** every 50 batches to the simulation cache (Section 4.5).

After the last batch, rows with a non-finite target or conditioning value are dropped (with a warning), the summary columns are winsorised, the standardisers are fitted, and the network is trained (Sections 5.3 and 6.4).

4.4 The susceptibility (χ) observation

A driven recording measures the bundle's linear response at one frequency. In χ mode the observation is the passive recording plus K such single-tone recordings, each turned into a complex number by a lock-in:

$$\hat{\chi}(\omega) = \frac{2}{F_0\, T} \sum_j \big(x_j - \bar{x}\big)\, e^{\,i\omega t_j}\, \Delta t_{\text{exp}}, \tag{9}$$

accumulated in double precision, with $\bar{x}$ the mean over the samples actually locked in, so $\hat{\chi}$ is in $\text{nm}\,\text{pN}^{-1}$ (`lock_in_batched`, `core/SBI/chi.py`). In training, each row proceeds as follows (`core/SBI/chi_probes.py`):

1. Ω_0 is the frequency of the periodogram peak of the row's *own* passive trace.
2. The probe count K is drawn once per batch from $\{2, \ldots, 12\}$, so the trained network accepts any count up to 12. (A simulated observation uses six.)
3. Probe j is driven at $f_j = m_j\, \Omega_0$. The multipliers lie in the band $(0.03, 0.3)$, below resonance; they are drawn by stratified jittering and then lifted, per row, into the sub-band that the row's own Ω_0 and recording length can resolve.
4. The drive has a *fixed non-dimensional* amplitude 0.15, i.e. a physical amplitude $0.15\, f_{\text{scale}}$, so lock-in signal-to-noise is uniform across the f_{scale} prior.
5. Each probe is locked in over a prefix of the trace: the full trace in 60 % of probes, otherwise a length drawn log-uniformly between 0.3 and 1 of it; the prefix is further capped at 20 drive cycles.
6. A probe with fewer than 2 locked-in cycles, a frequency at or above 0.9 of the Nyquist frequency, a band-normalised offset $|\hat{u}| > 1.25$, or a non-finite result is *masked*. A probe is never moved or silently dropped.
7. Each probe becomes six numbers: $u = \log(f/\Omega_0)$, $\log|\hat{\chi}|$, $\cos\arg\hat{\chi}$, $\sin\arg\hat{\chi}$, $\log(f\, T_{\text{locked}})$ and a mask. Valid probes are packed first, in ascending frequency; unused slots are exactly zero.

About 37 % of training probes end up masked (31.7–44.9 % across runs). The cause is almost entirely the two-cycle floor: Ω_0 spans about four decades over the prior, and slow cells have too few cycles in a 60 s recording. Masked probes are information that is genuinely absent, so the network is told which are live rather than being given placeholder values.

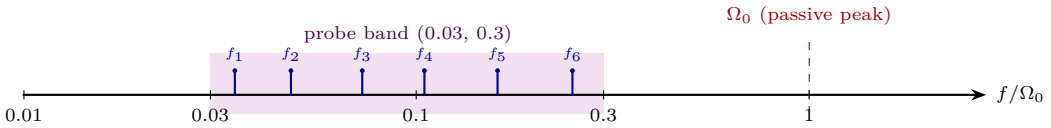


Figure 2: Probe placement in χ mode (schematic, $K = 6$). Each tick is one single-tone driven recording at $f_j = m_j\Omega_0$ with m_j inside the band; all lie below the resonance.

4.5 The simulation cache, memory and resumption

- The rows are accumulated in preallocated host tensors, and the training data stay on the CPU during the fit.
- The **simulation cache** (`Artifacts/simulations/`) commits batch by batch, every 50 batches by default. It stores the rows, the schedule, the initial conditions, the rotation V, snapshots of the random streams and the per-batch count of masked probes. A run can resume from it (`auto`, `never` or `require`); a mismatch of any compared setting refuses the resume and names the setting.
- The cache is keyed on the number of batches, the batch width, the prior's fingerprint, the chi constants and the rotation flag, not on the flow's hyper-parameters.
- A batch that runs out of GPU memory is retried at smaller widths, then after waits (3 attempts at 15, 60 and 180 s), and a learned cap remembers the widest batch that fit; an optional ceiling on the VRAM the run may use is set through `PRISM_VRAM_CEILING_GIB`. A full-size generation on the development card needs it.
- A *width versus count* trade-off: more rows per batch is nearly free for the solver, but the fit's time grows with the number of rows, and 5000×2048 and $10\,000 \times 1024$ are different training sets (the latter has twice the geometries).

5 What the network sees: the conditioning row

A conditioning row x is assembled in one place for every consumer (training, calibration, posterior-predictive check, simulated and experimental observations), so they cannot disagree about what a row means (`conditioning_rows`, `core/SBI/statistics.py`):

$$x = \big[\ \underbrace{s_1, \ldots, s_{41}}_{\text{statistics}}\ \big|\ \underbrace{v_1, \ldots, v_8}_{\text{validity flags}}\ \big|\ \log T_{\text{obs}}\ \big|\ \underbrace{\text{forcing or } \chi \text{ block}}_{4 \text{ or } 72}\ \big]. \tag{10}$$

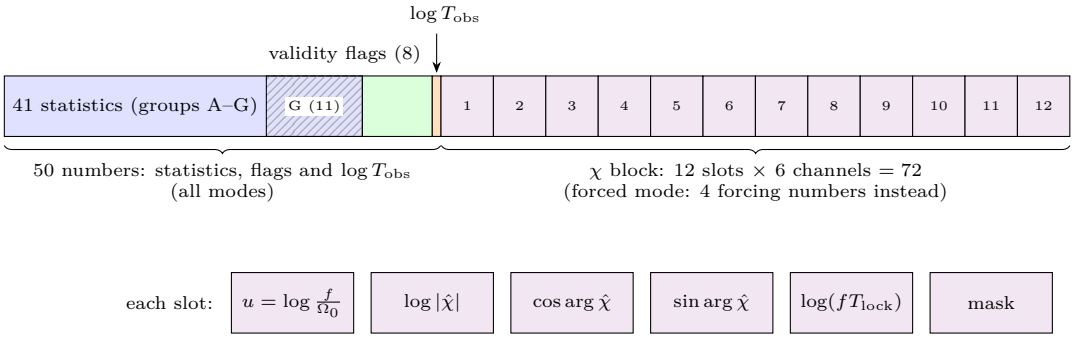


Figure 3: Layout of a conditioning row in χ mode (width $50 + 72 = 122$). Group G is all zeros except in forced mode. A padded slot is exactly zero in all six channels.

5.1 The 41 summary statistics

The statistics are computed on the downsampled, redimensionalised trace (Section 4.3). Positive quantities are log-transformed, phases are emitted as a cosine–sine pair, and non-finite values become 0 (`core/SBI/statistics.py`, `core/SBI/summaries.py`). Let f_{peak} be the periodogram peak frequency (DC excluded) and $A(t)$ the Hilbert envelope of the trace band-passed to 0.5–1.5 f_{peak}.

Table 4: The 41 statistics, in the order of `FEATURE_LABELS`.

Group	Features	How they are computed
A (4)	`A1_mean`, `A2_log_var`, `A3_log_fpeak`, `A4_log_acf_decay`	mean of the trace (the only statistic that is not demeaned: it anchors the dimensional level); log variance; log f_{peak}; log of the lag at which the autocorrelation falls to $1/e$.
B (9)	`B1_log_Q`, `B2_log_peak_floor`, `B3_log_centroid_ratio`, `B4_spec_entropy`, `B5_pow_frac_peakband`, `B6_low_freq_frac`, `B6_log_rolloff_ratio`, `B7_log_sec_freq_ratio`, `B7_sec_height_ratio`	spectral shape: log quality factor (peak over full width at half maximum, walked outward from the peak); log peak over median floor; log spectral centroid over f_{peak}; normalised spectral entropy; power fraction within 0.5–1.5 f_{peak}; power fraction below 0.5 f_{peak}; log 95 % roll-off over f_{peak}; log frequency ratio and height ratio of a secondary peak (it must exceed 5 % of the main one).
C (7)	`C1_log_norm_env`, `C2_log_env_cv`, `C3_env_onset`, `C4_mode_ratio`, `C5_env_skew`, `C6_log_slowenv_corrtime`, `C7_log_slowenv_relvar`	amplitude envelope: log mean envelope (normalised); log coefficient of variation; onset ratio $\langle A\rangle^2/\langle A^2\rangle$; mode over mean from a 64-bin histogram; skewness; log correlation time of the slow envelope (low-passed at 0.15 f_{peak}) times f_{peak}; log relative variance of the slow envelope.
D (3)	`D1_excess_kurt`, `D2_skew`, `D3_bimodality`	marginal distribution: excess kurtosis, skewness, Sarle's bimodality coefficient.
E (5)	`E1_log_tau_fast`, `E1_log_tau_slow`, `E1_w_fast`, `E2_log_h2`, `E2_log_h3`	multi-timescale and waveform: two decay times and the weight of the fast one, from log-autocorrelation slopes in two windows (ending at the crossings of 0.5 and 0.1); log power at $2f_{\mathrm{peak}}$ and $3f_{\mathrm{peak}}$ relative to the fundamental (±2 FFT bins).
F (2)	`F1_noniso_slope`, `F2_corr_omega_A2`	phase–amplitude coupling: slope of instantaneous frequency against A^2 (non-isochronicity) and their correlation.
G (11)	`G1_log_gain`, `G2_cos/sin`, `G3_log_h2_ratio`, `G4_log_h3_ratio`, `G5_cos/sin`, `G6_plv`, `G7_log_amp`, `G7_orient_cos/sin`	forced response, by lock-in at harmonics 1–3 of the known drive: log gain; phase relative to the drive; harmonic ratios; phase of the third harmonic relative to three times the first; phase-locking value; amplitude and orientation of the phase-binned residual variance. **All zeros in spontaneous and χ modes.**

5.2 Flags, duration and the mode-specific block

- **Eight validity flags** (`V_B1_Q`, `V_B7_secondary`, `V_C6_slowenv_tau`, `V_C7_slowenv`, `V_E1_tau_fast`, `V_E1_tau_slow`, `V_E2_h2`, `V_E2_h3`) say, per row, where a statistic was a substituted sentinel rather than a measurement. They are derived from the feature values, are part of x, and are deliberately excluded from the Fisher Jacobian.
- **$\log T_{\mathrm{obs}}$** is an input: the network is told how long the recording was, which sets the resolution of every statistic.
- **Forced block:** the four drive numbers (amplitude, frequency, phase, offset).
- **χ block:** the 12×6 packed probe set of Section 4.4.

5.3 Pre-processing before the network

1. **Filtering.** Rows with non-finite targets or conditioning values are dropped with a warning.
2. **Winsorising.** Every summary column (statistics, flags and $\log T_{\mathrm{obs}}$) is clipped to its own 0.1 and 99.9 percentiles, computed over the whole training set. The χ block is *never* clipped: clipping would push a padded slot off exact zero and turn it into a phantom probe.
3. **Standardising.** In χ mode the embedding network owns the standardisation (the flow's own z-scoring is switched off): the summary block is *rank-Gaussianised* (below), and the probe channels are standardised per channel over live probes only. In the other modes the library's per-column z-score is used.
4. **Layer normalisation** inside the networks (no batch normalisation anywhere, so a single-row inference equals the corresponding row of a batched one).

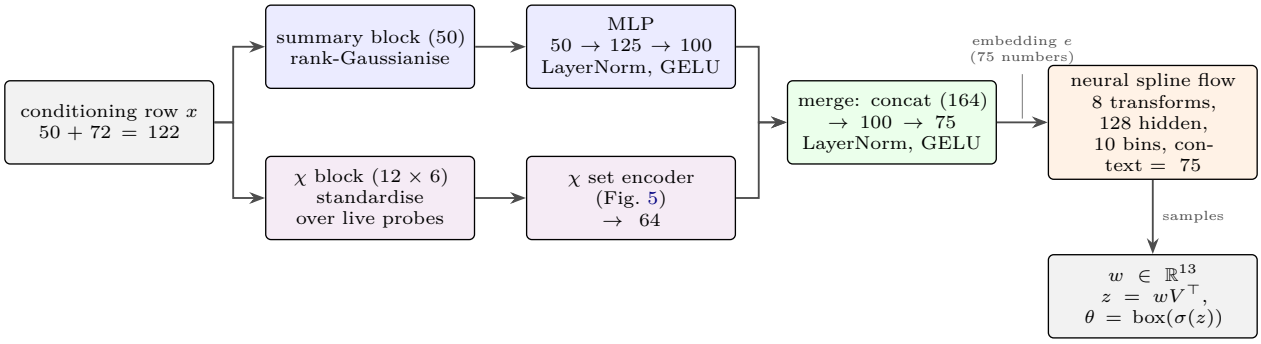


Figure 4: Network for χ mode, with the master box ($d = 13$). In forced mode the χ branch is a small dense branch ($16 \to 8$) on the four forcing numbers, and the summary block is z-scored by the flow's own standardiser; in spontaneous mode the summary MLP feeds a linear head to 75 outputs directly.

Rank-Gaussianisation replaces each value by the probit of its mid-rank in the training column, using the empirical quantile function at 1024 levels (ties map to the mid-rank; a constant column passes through as 0; the 11 group-G columns are such columns under χ). It was introduced because mean/standard-deviation standardisation had made two heavy-tailed statistics (`A1_mean`, whose standard deviation was 4.2×10^{11}, and `D3_bimodality`, 4.4×10^{8}) invisible to the flow: sweeping either across its physical range moved the embedding by less than one float32 unit in the last place.

6 The network

6.1 Architecture

The conditional density estimator has two parts: an *embedding network* that compresses a row into 75 numbers (`EmbeddedNet`, `core/SBI/embedded_network.py`; `core/SBI/chi_encoder.py`) and a *neural spline flow* [3] that models w given that embedding.

Component	Shape	Notes
summary pathway	$50 \to 125 \to 100$	each linear layer followed by LayerNorm and GELU; the width 50 is $49 + 1$, and the embedding width is $3 \cdot 50/2 = 75$
χ set encoder	$72 \to 64$	permutation-invariant, no batch norm (Section 6.2)
merge	$164 \to 100 \to 75$	LayerNorm and GELU after the first layer
flow	NSF, 8 transforms, 128 hidden units, 10 bins	context is the 75-number embedding; the planned retrain uses 256 hidden units and 10 transforms

Table 5: Component sizes for the master χ box.

6.2 The χ set encoder

The probe set has a variable number of live members in 12 slots, in no meaningful order, and its elements are samples of a smooth function $\hat{\chi}(u)$ at scattered abscissae. The encoder therefore must not depend on slot order and must not hide how much data there was.

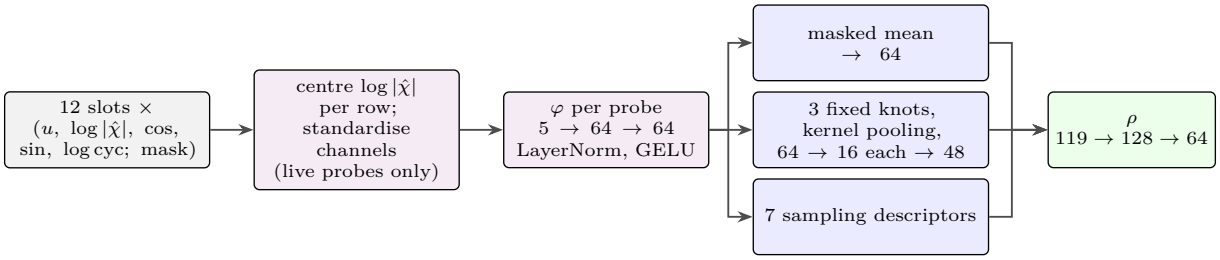


Figure 5: The χ set encoder: $64 + 3 \times 16 + 7 = 119$ pooled numbers enter ρ, which outputs 64 numbers.

- Each live probe is mapped by the same small network φ; padded slots are multiplied by the mask and contribute nothing.
- The **curve** summary is a masked mean of φ plus a Nadaraya–Watson quadrature at the fixed band-normalised knots $-1, 0, 1$ (Gaussian kernel width 0.6, shrinkage 0.5 toward zero where a knot is under-covered). Both approximate functionals of the susceptibility curve rather than of the sample.
- The **sampling** summary is deliberately dependent on the probe count: $\log(1+n_{\text{live}})$, the mean centred level, the mean u, the mean log cycle count and the per-knot coverage. An observation with two probes really is less informative than one with twelve, and the posterior conditioned on it should be wider. "Accepts any probe count" does not mean "returns the same answer".
- There is no max-pool: the expected maximum of n standard-normal draws shifts location with n (0.56, 1.27, 1.63 for $n = 2, 6, 12$), which would write a probe-count-dependent bias into every pooled channel.

6.3 The output: a flow on the latent coordinate

The estimator is sbi's `SNPE` with `posterior_nn(model="nsf")`, run with one round, which makes it plain amortised NPE [2, 4]. The flow is a neural spline flow [3]: a stack of coupling transforms, each a monotone rational-quadratic spline with 10 bins whose parameters come from a small network on the context e and the untransformed coordinates. It models the density of w (Section 2.4) conditional on the embedding $e = \text{embed}(x)$. After training, a `DirectPosterior` is wrapped in a `TransformedPosterior`, so that `sample` returns physical θ in the box and `log_prob` includes the Jacobian of the chain of Section 2.4. The output ordering is the bounds file's: the ten ND parameters, then the scale block.

6.4 Training

The network and the flow are trained jointly by maximum likelihood on the simulated pairs,

$$\mathcal{L}(\phi) \;=\; -\,\mathbb{E}_{\theta\sim p(\theta),\; x\sim p(x\mid\theta,t_{\text{scale}},T_{\text{obs}})}\big[\log q_\phi\big(w(\theta)\mid x\big)\big], \tag{11}$$

which is the NPE objective with the prior as proposal (so no proposal correction is needed). The optimiser is Adam with learning rate 10^{-3}, minibatches of 512 rows, early stopping after 20 epochs without improvement of the validation loss and no epoch cap. A non-finite loss raises an error. The planned retrain (10 000 batches $\times$ 2048 rows, a 256-unit, 10-transform flow) is budgeted at 672 card-hours and is followed by one certification narrowing round.

6.5 Why these choices (briefly)

- **Summaries rather than traces**: a 60 s recording is 60 000 samples; the model's information about the parameters is carried by a handful of spectral and temporal features, and these are cheap to compute on the GPU in bulk.
- **Rank-Gaussianisation** (Section 5.3) and **winsorising** protect the flow from heavy-tailed statistics.
- **The Fisher rotation** (Section 3.4) removes oblique ridges from the latent posterior; it adds no information.
- **The χ set encoder** lets one network accept 2–12 probes at any frequencies inside the band.

7 Calibration, informativeness and predictive checks

A trained flow $q_\phi(\theta \mid x)$ is a claim: that it is the posterior. PRISM tests the claim on fresh simulations, never on training rows, which the flow has already fitted and which would flatter it. Four different questions are asked, and no check answers another's (Table 6).

7.1 The calibration set

`validate` simulates $n_{\text{cal}} = 2000$ datasets. Each draws a true parameter vector θ^* from the posterior's own training prior (for a narrowed posterior, the prior restricted to its region), simulates an observation with the same generator as training (Section 4.3), including the χ probes, whose count is drawn per batch over the mixture training saw, and then draws $L = 1000$ posterior samples. The datasets are spread over 200 $(t_{\text{scale}}, T_{\text{obs}})$

Table 6: What each check asks, and what it cannot tell you.

Check	Question	Run by	Cannot tell you
Rank test (SBC)	Is each parameter's marginal calibrated: neither over- nor under-confident, nor biased?	`validate`, `sbc`	Whether the posterior learned anything: a posterior that returns the prior is flat by construction.
Joint coverage (TARP)	Is the joint posterior calibrated?	`validate`	A flaw in one parameter of 13; the test has little power against one. A p-value of 1.000 is not "perfectly calibrated".
Informativeness	Did the run learn anything at all?	`validate`	Whether the posterior is calibrated.
Predictive check	Do simulations from the posterior reproduce this observation's statistics?	`infer`	Whether the posterior is too wide: an over-dispersed posterior passes.
Identifiability diagnostics	Is the information in the data at all, and can the flow see it?	`identifiability`, `ablation`, `probes`	Anything about a particular posterior's calibration; they measure and change no setting.

operating points: every dataset in a batch shares its batch's t_{scale}, because the generator overwrites it per batch (Section 4.3, step 3). Every draw except the χ probe layout runs on a random stream derived from the seed and a fixed calibration tag, so a calibration never replays the stream a training run with the same seed started on; `validate -seed` repeats one.

7.2 Rank uniformity (SBC)

For each dataset, the rank $r_i = \#\{\ell : \theta_i^{(\ell)} < \theta_i^*\}$ of the true value among the posterior draws is uniform on $\{0, \ldots, L\}$ whenever the posterior is calibrated [5]. Each parameter's ranks are tested against the uniform distribution with a Kolmogorov–Smirnov (KS) test, and a parameter passes when $p \geq 0.05/d$, where d is the number of inferred parameters (12 or 13, temperature included where it is inferred). Dividing by d keeps one 5 % false-alarm rate across all of them, and temperature is judged like the rest: its rank test is valid whether or not the data inform it. Two reading rules matter:

- Read the KS p-values, not the `c2st_ranks` score printed beside them: about 0.58 is that score's finite-sample floor, so a value near it says nothing.
- Read the pooled rank histograms beside KS. At 2000 datasets KS flags a mild miscalibration reliably, and the histogram says how severe it is; a near-flat histogram under a low p means a mild one.

7.3 Joint coverage (TARP)

TARP [6] draws a reference point, finds the fraction of the posterior draws that lie closer to it than the truth does, and compares the distribution of those fractions with the uniform one: for a calibrated posterior the coverage curve lies on the diagonal. PRISM fixes the curve at 30 bins, because the test compares the binned curve with the diagonal point by point, so more bins make the same miscalibration fail harder and a bin count that grew with the number of datasets would tie the pass mark to it. The test passes at KS $p \geq 0.05$. Since sbi 0.26 the samples, the truths and the reference points are scaled to the range the calibration parameters span, so no parameter dominates through the size of its raw values; its area-to-curve statistic is the signed area between the coverage curve and the diagonal over the upper half of the grid. TARP figures recorded before 1 October 2026 were made under sbi 0.25 and are not comparable with later ones.

7.4 Informativeness

Calibration is a property of the joint distribution, so a flow that returns the prior passes every test above. The number that says whether a run learned anything is the expected prior-to-posterior gain, estimated on the calibration set just simulated, so it costs no further simulation:

$$\widehat{I} \;=\; \frac{1}{n_{\text{cal}}} \sum_{j=1}^{n_{\text{cal}}} \Big[\log q_\phi(\theta_j^* \mid x_j) - \log p(\theta_j^*) \Big] \quad \text{(nats)}. \tag{12}$$

- **Read its sign first.** It is not bounded below by zero: a flow that gives the truth less density than the prior scores negative (a smoke-sized training measured −23.1 nats, the right answer for it).
- **It is joint**, over every parameter together, temperature included. The per-parameter breakdown is not a set of KLs: those figures are entropy reductions estimated from draws (how much narrower each marginal got), so a marginal that moves without narrowing scores 0 and a widened one scores below 0.
- **Compare only on fresh calibration sets.** A figure measured on training rows is optimistic by an unknown amount.
- For a narrowed posterior the total is still measured against the full prior and is inflated by $-\log P(A)$ nats, A being the region (Section 9); the amount is printed, and nothing subtracts it.

7.5 The verdict, and how to read it

The calibration verdict

PASS when every parameter's rank test reaches $p \geq 0.05/d$ *and* the joint coverage test reaches $p \geq 0.05$. A FAIL is a result, not an error: the calibration is written and returned all the same.
Every verdict carries one caveat. t_{scale}'s rank test rests on the 200 operating points, not on the 2000 datasets, so it has less power than the others.
A flat SBC means "not overconfident", never "informative". An earlier χ posterior was flat on 12 of its 13 parameters, had a TARP p of 1.000, and had every non-dimensional marginal at the prior.

Pooled calibration over mixed probe counts. In χ mode the rank test pools across probe counts, and a pooled test can be flat while each count is miscalibrated in compensating directions. `sbc -chi-k-fixed K` holds the count, one stratum at a time, and `sbc -repeats N` repeats the rank test on fresh calibration sets, to tell a lucky draw from a real miscalibration by how far the p-values vary from run to run (it does not run the coverage half).

7.6 Diagnostics that ask whether the information is there

These commands measure a posterior or a cell and change no setting; each writes one record into the store.

7.7 What the August 2026 retrain showed

The last full χ retrain before the conditioning repair (5000 batches × 2048 rows, 10.24 million simulations; its artifacts no longer exist, so the numbers stand as a record) is a good illustration of why the checks differ. Calibration was excellent: flat rank histograms on all 13 parameters and TARP on the diagonal. Prediction was over-dispersed: the predictive check covered 99.1 % at a nominal 90 % (mean $|z| = 0.490$), with a 95 % band two to three times the observation's envelope: honest intervals, and far too wide. The train and validation losses tracked each other and plateaued from about epoch 90, which reads as an identifiability limit in the data and not as under-fitting. (That reading was made on a flow whose conditioning could not see two of its own channels; the planned retrain uses the repaired conditioning and a larger network, so the plateau is to be read afresh.)

8 Running the posterior on an observation

8.1 What an observation is

An observation is a conditioning row (Figure 3) built exactly as the training rows are, in the mode the posterior was trained in. A posterior loads only in its own mode, and the mode and the row's width are checked when the observation loads.

The three modes of Table 1 ask for different recordings and notes. *Spontaneous:* one passive recording and its length. *Forced:* a passive recording and one driven recording of the same length, with the drive noted as one value per forcing parameter in SI units (force in newtons, frequency in hertz, phase in radians); keep it inside the bounds file's Forcing box, because nothing checks a recording's drive against it. *χ:* a passive recording and up to 12 single-tone driven recordings, with the frequency each was actually driven at (hertz) and the physical

Table 7: The diagnostic commands.

Command	Reads / simulates	What it answers
`identifiability rotation`	the posterior's record; simulates nothing	Decomposes the Fisher eigenbasis: eigenvalue spread, participation ratio, directions best-constrained first with their largest loadings, and each parameter's share of identifiability in the worst directions.
`identifiability laplace`	simulates at the cell's ground truth and at prior draws (forced mode only)	The Laplace marginal standard deviation of every parameter there: whether the data could pin it down, whatever the posterior learned.
`identifiability jacobian`	simulates at the cell's ground truth; no posterior (Nadrowski only)	The degeneracy map: how strongly each parameter moves the features the posterior conditions on, in units of the feature noise, and which parameters move them alike. In χ mode it first prints the probe budget (drive cycles per probe, shortest and longest usable recording).
`ablation`	the posterior and its simulation cache; simulates nothing	Which conditioning channels the flow can see: each summary channel is swept from the 1st to the 99th percentile of the rows it trained on, and the embedding's displacement is graded healthy, compressed, severely compressed, invisible, constant in training or non-finite.
`probes band`	simulates the cell, driven and undriven	Do the configured χ band and drive hold for this cell? Four criteria: spread of $\lvert\chi\rvert$, phase scatter, signal over the same lock-in on undriven runs, and capture (the share of the cell's own peak power that survives the drive). Only capture is physical evidence; the other thresholds are conventions.
`probes mask`	a few batches of the training generator over a prior	Why training probes are thrown out: too slow for the cycle floor even at the band's top, shortened below the floor by the duration draw, or a lock-in that came back non-finite or zero.
`probes drive`	simulates the cell undriven, then driven	How hard a lab can drive this cell: the strongest drive at which the bundle still oscillates freely and the weakest at which the drive captures it.

drive amplitude (newtons), one value shared by every probe. The lock-in divides by the amplitude it is given, so give the amplitude actually applied: training drove every probe at 0.15 times the cell's force scale (1.5 pN on the master cells), an active bundle does not respond linearly, and a much stronger drive can capture the bundle, which then follows the drive instead of its own rhythm.

8.2 Simulated and measured observations

Simulated. `infer -cell` re-simulates a cell file's ground truth at the chosen T_{obs} with the same fine step and downsampling as training (`core/orchestrator.py`, `generate_observations`). In forced mode the cell's own drive is used; in χ mode the probes sit at the configured multiples of the cell's own peak frequency and the drive in the file is ignored. A simulated observation has a known truth, so the run can flag a truth outside the training distribution and mark the truth on the plots.

Measured. Recordings are read by `build_experiment_observation` (`core/SBI/observations.py`); the rules are strict because nothing downstream can detect a misread frequency:

- A recording is a `.npy` file with one 1-D array, or a `.csv` whose last column is the trace; it must be sampled at 1 kHz (the training step, `DT_EXP_S`), in the model's length unit (nm for the Nadrowski cell), and the passive recording should hold $T_{\mathrm{obs}} \times 1000$ samples. In forced mode both recordings must have the same length; in χ mode each driven recording may have its own and is locked in over its length up to the ceiling.
- Frequencies are given in hertz and converted to the cell's own unit. Give the frequency you actually drove at: the lock-in answer decays like a sinc function of the error, so an error of a fraction of $1/T_{\mathrm{obs}}$ spoils the estimate while every number still looks plausible.
- In χ mode Ω_0 is measured from the passive recording, and each probe is judged by one rule set: *masked* if it has fewer than two drive cycles (it stays in the set and contributes nothing, as in training), *truncated* if it has more than 20 (only the first 20 cycles are used), and *refused* for a frequency that is not finite and positive, one at or above 0.9 times the Nyquist frequency (450 Hz at 1 kHz), one outside the band judged against this recording's own Ω_0 (about 0.0225 to $0.4\,\Omega_0$ is accepted), a passive recording with no usable

peak, more probes than slots, or a set in which no probe survives.

The shortest useful recording. The band's edges are ten times apart, and so are the 2-cycle floor and the 20-cycle ceiling. The lowest probe ($0.03\,\Omega_0$) first reaches two cycles at $T^* = 2/(0.03\,\Omega_0) \approx 66.7/\Omega_0$ seconds (Ω_0 in hertz). Below T^* the lowest probes are masked; above it the highest are truncated, which costs nothing. A recording longer than T^* is therefore better, and the training range (Section 4.3) bounds it from above: about 7.4 s is inside it at every t_{scale} on the master box, and 26.9 s at the master cells' t_{scale} of 3.73.

8.3 Checks before sampling

Before the posterior is asked anything, the run compares the observation with what the posterior was trained on.

- A T_{obs} outside 1–60 s warns, and so does a simulated observation longer than training holds at the cell's own t_{scale}.
- A simulated truth outside the 1st–99th percentile of draws from the training prior warns that the posterior will extrapolate and may simply revert to the prior for that parameter. This is a sampling check, not a bounds check, because the prior is stability-screened: a value can sit inside the box yet in a corner the training data never visited. Phases are skipped (they are circular), and in χ mode the cell's own drive cannot be out of distribution because it is ignored.
- A narrowed posterior refuses any observation but its region's own unless told otherwise (the `Accept` option), and a truth outside its region warns.

8.4 Sampling and outputs

The flow draws `-n-samples` = 1000 samples in the rotated latent coordinates; each is carried back through the chain of Section 2.4 to physical parameters that cannot leave the box. A run writes two records, an observation and an inference, and produces:

- the per-parameter summary (5 %, median, 95 %) and, for a simulated cell, the ground truth beside it;
- the corner plot of the posterior;
- the posterior predictive check (below) and the "best fit" table.

Posterior predictive check. Every posterior draw is simulated at the observation's geometry (its T_{obs}, the draw's own t_{scale}; draws are sorted by t_{scale} and simulated in bins of 50, so each bin shares a similar fine grid) and turned into a conditioning row. Each observed statistic is then compared with its simulated cloud (`core/SBI/ppc.py`, `core/SBI/analysis.py`):

$$z_i = \frac{s_i^{\mathrm{obs}} - \bar{s}_i}{\mathrm{sd}(s_i)}, \qquad \mathrm{coverage}_{90} = \frac{1}{N_s}\sum_i \mathbb{1}\left[q_{05}(s_i) \le s_i^{\mathrm{obs}} \le q_{95}(s_i)\right], \tag{13}$$

with mean and maximum $|z|$ reported. Statistics with zero variance across the draws are counted as invalid and split by origin, not scored. Read the check as "the observation is typical of what the posterior predicts", never as proof that the posterior is right: a wide posterior passes by being wide.

The "best fit" table is not a recovery measurement

It reports the posterior draw whose simulated summary statistics lie closest to the observation's. Each feature is standardised by its spread across the posterior's own draws, not by measurement noise, so the score says only that the draw sits inside the predictive cloud. It weights every live feature equally, so along a degenerate direction the best draw is close to a free draw. Trust the table only where a marginal is sharp; elsewhere read where the truth falls in the marginal.

9 Narrowing rounds (TSNPE)

An amortised posterior is trained once for every observation the box can produce. When one observation matters, a *narrowing round* (truncated sequential neural posterior estimation, [7]) spends a new simulation campaign near that observation only: it draws a region around the observation along the posterior's Fisher

directions, trains a new posterior on the prior restricted to that region, and is valid only near that observation (`core/SBI/truncate.py`, `tsnpe_round` in `core/orchestrator.py`).

9.1 Draw from the truncated prior, never from the posterior

The posterior only says where to look; the training proposal is the prior restricted to the region. Restricted to a region, the proposal is the prior up to a constant, so the round needs no proposal correction. Proposing from a density fitted to the posterior would multiply the likelihood in again every round and temper it: credible intervals would contract as $(L+1)^{-1/2}$ after L rounds with no new information (2.2 times narrower than the data support at $L = 4$), and SBC would still come out flat, because it validates the flow against the proposal it trained on. A test pins the proposal's width to the prior's over the region, neither the posterior's nor the posterior's over $\sqrt{2}$.

9.2 How a round runs

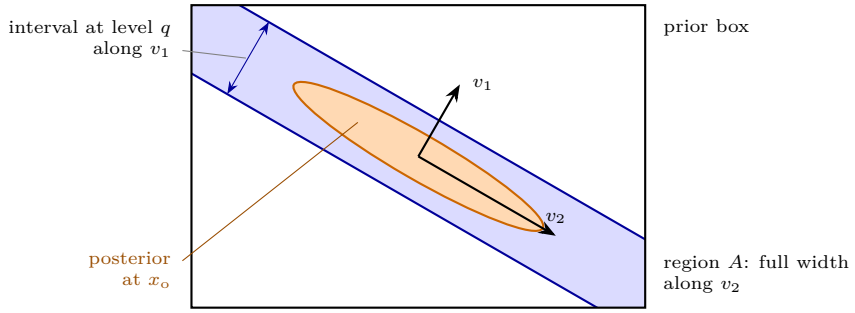


Figure 6: A narrowing round in two parameters. The orange ellipse is the parent posterior at the observation; v_1 and v_2 are its Fisher directions, v_1 the best constrained. The region A (blue) is an interval at level q along v_1 and the full prior width along the flat direction v_2; the new posterior is trained on the prior restricted to the blue strip.

1. **Inputs.** A parent posterior and an observation (referred to by its record; the observation must match the stored one). Unweighted posterior draws are taken at the observation: not best fits, because choosing the best fits would apply a second, undeclared likelihood with the discrepancy measure as a hidden setting.
2. **The region.** The draws are carried into the parent's Fisher eigenbasis ($w = zV$), where the posterior is close to axis-aligned. For each of the leading $k = 5$ directions (`-directions`) the interval holding $q = 99.9\,\%$ of the draws (`-level`) is taken; the remaining directions keep the full prior width. A box in the physical parameters would cut barely constrained ones on noise, and deleted support never comes back.
3. **Skip rule.** A direction whose loading on t_{scale} exceeds $1/\sqrt{d}$ (0.277 at $d = 13$, the root-mean-square entry of a random rotation) is left at full width and the next eligible direction is truncated instead; on the card run's tier-1 round the first direction loaded 0.960 on t_{scale} and was skipped. The reason is that every training batch overwrites its rows' t_{scale} with the batch's value after the draw (Section 4.3): along a direction that loads on t_{scale} that carries rows out of the box, and the proposal becomes a smooth reweighting $p(\theta)\,P(A \mid \theta_{-t})/P(A)$ that NPE does not correct.
4. **Campaign.** Parameters are drawn from the parent's prior restricted to the region (rejection sampling), a new simulation campaign is run through the same pipeline (default 5000 batches), and a fresh flow is trained. The region is part of the cache's identity, so a round never resumes an amortised run's rows. A round does not inherit the parent's network size or batch count.
5. **Calibration.** The flow converges to the posterior under the *restricted* prior, so calibration draws from that prior, and a flat result certifies calibration on the region only.

9.3 The safety rules

9.4 Reading a round

- The region is a box of per-direction intervals at level q over k directions, so its mass has a *bound*, not an equality: by the union bound it holds at least $1 - k(1-q)$ of the posterior's mass, 99.5 % for five directions at 99.9 %.

Table 8: The narrowing-round rules and what each prevents.

Rule	What it prevents
Truncated prior	Tempering: a round draws from the prior restricted to the region, never from the posterior.
Observation digest	A region drawn around data that are not verifiably the recorded ones; it deletes prior support for good.
Narrowed model	A narrowed posterior loading or inferring as a broad one; its flow saw rows only inside the region, so elsewhere it extrapolates.
Eigenbasis	Cutting barely constrained physical parameters on noise: the region is cut in the leading Fisher directions, flat ones left full width.
Unweighted draws	A second, undeclared likelihood from choosing best fits.
Generous region	A region too tight: 99.9 % by default, and a level below 0.99 raises a pre-flight warning. Too wide costs only simulations.
Cost on screen	A surprise bill: a round is a full simulation campaign, so its budget is shown before it starts.
Region carries its basis	A box that means nothing: the child reuses the parent's rotation, never recomputes it, and names its base prior.
Calibrate on the region	Reading a flat SBC on the wrong prior.

- For an observation simulated from a cell the round reports, for every truncated direction, whether the truth lies inside the interval, and records it in the new posterior's `training.truth_containment`. It also prints the fraction of prior draws the region accepted and how many training targets lie inside it after the t_{scale} override.
- A narrowed posterior's informativeness is measured against the full prior and is inflated by $-\log P(A)$ nats (Section 7). The amount is printed beside the kept fraction and is not corrected.
- A narrowed parent drawn around another observation is refused; start from an amortised posterior, or draw the round around the parent's own observation.

10 Artifacts, reproducibility and operation

10.1 The artifact store

Everything generated lives under `Artifacts/<kind>/<name>__<id>/`, a folder with a `manifest.json` and, for every kind except the simulation cache, a `log.txt` of the run's records. The hand-edited inputs live apart, under `Resources/` (bounds, cells, units, models); the bounds file declares which parameters are inferred and in what order, and therefore the observation mode.

Table 9: The kinds of record in the store.

Kind	What it holds
priors	the stability-screened prior (Section 3.2)
simulations	the training cache: rows committed batch by batch under an identity digest; it has no writer of its own
posteriors	the trained flow with its network, the conditioning layout, the transform keys and bounds, the Fisher rotation and its eigenvalues, the mode, the χ constants, and whether it is amortised or narrowed
observations	one conditioning row, simulated or measured, with its provenance
calibrations	the calibration set's rank and coverage results, the verdict and the informativeness
inferences	the draws and the 5/50/95 % summary for one observation (with the ground truth for a simulated cell)
diagnostics	one record per `sbc`, `identifiability`, `ablation` or `probes` run
fdt	the effective-temperature measurement and its parameter sweeps (outside this workflow)

- **Stages write at completion.** A save is a rename. There are two exceptions: the training cache, which commits batch by batch and has no writer at all, and an `fdt` record, which is progressive.
- **Loading checks.** A loader refuses a mismatch in what it compares: the bounds file, the mode, the conditioning width, the transform and a χ posterior's band, drive, lock-in ceiling and slot count. The escape hatches are `Accept(truncated, other_observation)`, and each use is recorded downstream.

- **Lineage.** Records name their parents; `python -m core artifacts summary inference NAME` prints an inference's lineage back through its observation and posterior to the prior and the simulation cache.
- **Provenance.** A record's `env` block names the library versions (a TARP figure is only comparable with another made under the same sbi), and a posterior stamps the checkout's git revision and dirty flag when it is written, so a commit made during a long run moves that stamp.

10.2 Reproducibility

Every knob is an argument, never a write to the configuration module: the stages bind the defaults at import, so assigning to them at run time would silently change nothing. A public stage copies the configuration it is handed, so a caller reads what a stage wrote from the artifact. Seeds are explicit (`-seed`); a calibration derives its stream from the seed and a fixed tag. Two caveats: the Fisher rotation is *not* reproducible from one process to the next, because its operating points come from unseeded random streams (which is why a narrowing round inherits its parent's rotation); and every repeat of a calibration draws the same χ probe layout, so repeats do not vary that part.

10.3 Hardware, memory and long runs

- Simulation runs on the GPU when one is present, as CUDA graphs that replay 50 Euler steps, in segments of at most 100 000 fine steps; the summary statistics run in sub-batches of 256 rows. A planner sizes each batch against the free video memory, and the environment variable `PRISM_VRAM_CEILING_GIB` caps it: on the reference card (16 GB shared with the desktop) a full-size generation needs the ceiling set. A batch that runs out of memory is retried (three attempts, with 15, 60 and 180 s pauses).
- The simulation cache commits every 50 batches, so a stopped generation resumes from it. **A fit is not checkpointed:** any stop during the fit loses the whole fit, while the simulated rows survive in the cache.
- The planned full-size retrain is a χ posterior on the tier-1 box: 10 000 batches of 2048 rows (20.48 million rows), a flow of 256 hidden features and 10 transforms, then one narrowing round that certifies it, within a budget of 672 card-hours. Runs of this length are meant to be started from the window, and they need the card to themselves: no test run, no second window.

10.4 Front ends

Two front ends drive the same stage functions. The window (PySide6) opens on Home; its *Parameter Inference* section has six tabs that share one session, and a tab is greyed until the session holds what it needs. The command line is `python -m core <subcommand>` (`-help` lists them); every flag maps onto a stage argument, and every refusal is a rule that names the offending control in the window and the offending flag on the command line.

Table 10: The stages, where to run them, and what each writes.

Stage	Window tab	Command	Writes
Model-level choices (mode, χ, rotation)	Config	flags shared by all	nothing
Build the prior	Prior	`prior`	a prior
Train	Posterior	`train`	the simulation cache and a posterior
Calibrate	Validate	`validate`	a calibration
Infer	Infer	`infer`	an observation and an inference
Narrow	TSNPE	`tsnpe`	a narrowed posterior
Diagnostics	(command line only)	`sbc`, `identifiability`, `ablation`, `probes`	diagnostics
Whole chain, tiny	(command line only)	`smoke`	every stage at small size
Browse and manage records	Artifacts screen	`artifacts` (`list`, `show`, `note`, `rm`, `sweep`, `summary`)	notes only

A stage chain at small sizes (it shows the commands working and means nothing more; the first line points the store at a scratch folder so the real one gains nothing):

```
export PRISM_ARTIFACTS="$HOME/prism-scratch"
python -m core prior    --chi --bounds Resources/Bounds/nadrowski/master.txt --name demo_prior
python -m core train    --chi --bounds Resources/Bounds/nadrowski/master.txt --prior demo_prior \
                        --num-runs 8 --run-size 32 --max-epochs 5 --name demo_posterior
python -m core validate --chi --bounds Resources/Bounds/nadrowski/master.txt \
                        --posterior demo_posterior --n-cal 100 --seed 7
python -m core infer    --chi --bounds Resources/Bounds/nadrowski/master.txt --posterior demo_posterior \
                        --cell Resources/Cells/nadrowski/master_spont.txt --t-obs 4.5 --name demo_inference
python -m core artifacts summary inference demo_inference
```

11 What the data can say, and what is still open

11.1 Identifiability

Three parameters are seen only through noise. In the Nadrowski model n, `temp` and `tau_c` appear in no drift term: n enters all three noise amplitudes, `temp` only the adaptation row's and `tau_c` only the calcium row's, and the observable is the bundle's displacement. The data therefore see them only as latent noise propagating into the displacement, and no retraining, feature repair or reparameterisation gives them a handle the observable lacks. On the tier-1 box, in forced and χ mode, n gains a second route through the derived force scale, which it shares with the bath temperature T as the product $n \cdot T$, and T itself is only weakly informed (nearly degenerate with n).

What χ buys. The passive and forced traces see two products, one setting the amplitude and one the timescale; $\chi(\omega)$ over frequency was meant to separate k, `lam`, x_{scale} and t_{scale} one by one. Measured with the degeneracy map (Table 11), part of that holds. A unique-handle score is the fraction of a parameter's feature gradient that the other parameters' gradients cannot explain: 1 alone, 0 fully aliased.

Table 11: Unique-handle scores on the master cell at $T_{\mathrm{obs}} = 4.5\,\mathrm{s}$ (forced mode on a weak-drive cell against χ mode with six probes; the 1.0 s column is a control on the forced lock-in).

Parameter	forced, 4.5 s	χ, 4.5 s	forced, 1.0 s (control)
`k`	0.040	0.091	0.076
`x_scale`	0.043	0.125	0.082
`t_scale`	0.219	0.447	0.206
`lam`	0.278	0.472	0.270
`delta_E`	0.091	0.102	—
`f_scale`	0.871	0.695	—

- The `t_scale`, `lam` and `f_scale` gains are χ's real product, and the control leaves them untouched. The phase channels carry `lam` and `f_scale`; the magnitude channel carries `t_scale`.
- `f_scale`'s score falls although its handle improves: a score is a ratio, and a parameter with no gradient is trivially unique. Forced mode barely measures `f_scale` at a weak drive (gradient norm 0.018), so its 0.871 is noise being unique; under χ the norm is 3.789, 213 times larger. Read a score beside its gradient norm, and compare only maps made at the same T_{obs}.
- k and x_{scale} stay degenerate (gradient $|\cos|$ 0.98 forced, about 0.95–0.97 under χ): stiffness and displacement scale move the trace's mean together, and a sub-resonance susceptibility does not touch the mean. χ helps them a little, within the effect of recording length, and no retraining will change that.
- The Fisher rotation is measured on clean features (a Jacobian standardised by feature noise measured locally), so the identifiability findings are not artifacts of the conditioning repair: a gradient small against trajectory noise is small whatever the flow can see. A clean loss plateau well before the best epoch is read as an identifiability limit, not under-fitting, with one exception: for t_{scale}, T_{obs} and the probe design, whose effective sample size is probably the batch count, a plateau may say nothing about convergence.

11.2 Approximations and leads in the pipeline itself

A lead on the time scale (reproduced for an ideal response, not yet audited end to end)

The simulation runs on one fine grid whose step is the frame interval over the largest time scale in the box, and the response is put on the recording's clock by keeping every m_k-th fine point, with $m_k = \max\big(1, \mathrm{round}(t^{\mathrm{hi}}_{\mathrm{scale}}/t^{(k)}_{\mathrm{scale}})\big)$ a whole number. Every later step treats the kept points as exactly Δt_{exp} apart, which is true only when $t^{\mathrm{hi}}_{\mathrm{scale}}/t^{(k)}_{\mathrm{scale}}$ is an integer. Otherwise the simulated trace is that of a cell with time scale $t^{\mathrm{hi}}_{\mathrm{scale}}/m_k$ while the row is labelled with the continuous $t^{(k)}_{\mathrm{scale}}$: at a label of 25 ms the trace is a cell at 20 ms, at 30 ms a cell at 40 ms, and at the master cell's 3.73 ms a cell at 3.64 ms. The drive is built with the continuous value, so it is detuned from the lock-in that measures the response, and a measured $|\chi|$ is attenuated by an amount that depends on t_{scale} and on how many drive cycles are locked in. A bench recording has no such detuning.

What has and has not been shown: the arithmetic of the real functions was reproduced for a system whose response equals its drive; no end-to-end training batch has been run, the size of the effect on a real bundle is unknown, no fix has been designed, and a calibration on simulated data cannot see it, because the calibration set comes from the same simulator. It may be what the 20-cycle lock-in ceiling works around, but that link is an untested inference. Two further leads, a drive phase carrying the time of the discarded transient and a solver step that may drift between segments, were not reproduced.

Smaller approximations, each stated in the code or the guide:

- **The prior's walk step** is 0.01 in every parameter's own units whatever its range (from 2 for `tau` to 290 for n), so on the built-in boxes the walk adds near-copies of the census's points rather than exploring: the accepted set is the census, re-weighted slightly. That is harmless where nearly the whole box is stable, and it matters for a model whose stable region is thin.
- **The census is linear in the box.** A per-parameter log box changes only the coordinate the mixture is fitted in, not where the census puts its candidates.
- **The solver's finest step** comes from a prior bound, not from the dynamics; the convergence study that would settle it (halve the step until each of the 41 features, in units of its own noise, stops moving) has not been run.
- **Effective sample size.** Every row of a batch shares one t_{scale}, T_{obs} and probe set, so along those axes the effective sample size is probably the batch count, not the row count.
- **Fisher rotation.** It is averaged over eight operating points drawn from unseeded streams and therefore not reproducible between processes.

11.3 Open questions

Each is unknown today; each says what would settle it.

- **M-replicate conditioning, first.** A posterior's width mixes a real degeneracy with the sampling noise of statistics computed from one realisation. Simulating M independent passive traces per parameter set and conditioning on them with a permutation-invariant encoder like the probes' would separate the two: widths falling as $M^{-1/2}$ mean estimator noise, which longer or repeated recordings fix; widths that saturate mean a real degeneracy, which only a new observable reaches.
- **The thermal tail.** Above the bundle's corner frequency the displacement spectrum tends to $2\,x^2_{\mathrm{scale}}/(n\,\beta\,t_{\mathrm{scale}}\,\omega^2)$, an absolute reading of those scales that no feature takes (every spectral feature is a ratio or a fraction). One new feature, the mean of $\log(\omega^2 S_X)$ over about a decade above the corner, would take it.
- **The strata test.** Train 5000 × 2048 and 10 000 × 1024 (the same rows over 5000 and over 10 000 operating points) and compare t_{scale}'s rank test and width.
- **New observables** (only after the replicate study): a step or force-clamp transient for k, and two-tone intermodulation at $2\omega_1 - \omega_2$ for the parameters that shape the nonlinearity.
- **Probes against slots.** A simulated observation supplies six probes into twelve slots; whether more probes, or a slot count matched to what is supplied, would buy information has never been measured.
- **Tiers 2 and 3, and fixing T.** Tier 2 would take x_{scale} from the instrument's calibration; tier 3 would restrict the prior to the Ω_0 band a preparation actually produces. The alternative to tier 1's inferred temperature is to fix T at 300 K and infer 12 parameters.

- **Chi probe design** still has three measurements open: the band's top edge (0.3), the rows below 0.3 Hz, and the wander of the bundle's response over tens of drive cycles.
- **Narrowing rounds:** pooling rows across rounds, the batch-by-scale t_{scale} override that stops t_{scale}-loaded directions from being truncated, and any change to how the Fisher eigenbasis is built.

A Defaults by stage

Every number below is a default in `core/config.py` (or a flag default of the command-line tool). Each is an argument of the stage that uses it: the window and the tool pass them explicitly, and a run records the ones it used in its manifest. The window opens every science knob at these values on every launch.

Table 12: Defaults by stage. `config.py` constants in the first column; `-flag` names are the tool's.

Setting	Default	What it controls
Recording and solver		
`DT_EXP_S`	1 ms	recording interval (1 kHz frames)
`T_MIN_EXP_S`, `T_MAX_EXP_S`	1 s, 60 s	shortest and longest recording a run accepts
`TRANSIENT_ND_UNITS`	100	transient discarded before the record (4000 fine steps at the default step)
`N_ND_MAX`	300 000	largest number of fine steps in one batch
`CHUNK_LEN`	100 000	fine steps per integration segment
`SOLVER_CUDA_GRAPHS`, `SOLVER_GRAPH_CHUNK`	on, 50	CUDA-graph replay, in blocks of 50 Euler steps
Prior		
`STABILITY_SWEEP_ND_UNITS`	1000	ND time units each candidate is integrated for in the stability screen
`PRIOR_SWEEP_ITERATIONS`	50	rounds of the Sobol census (candidates screened = batch × rounds)
`PRIOR_SWEEP_BATCH`	0	candidates per round (0 = the hardware batch)
`PRIOR_SWEEP_MAX_SETS`	175 000	accepted sets that stop the random walk (the point cloud the mixture is fitted to)
`PRIOR_SWEEP_STEP`	0.01	standard deviation of a random-walk step, in each parameter's own units
`PRIOR_CLUSTER_MIN_SIZE`, `PRIOR_CLUSTER_MIN_SAMPLES`	50, 10	HDBSCAN settings; the labels set only the number of mixture components
Training data		
`TRAINING_NUM_RUNS`	5000	training batches (`-num-runs`)
`TRAINING_RUN_SIZE`	0	ceiling on simulations per batch (0 = follow the device batch size)
`TRAINING_CHECKPOINT_EVERY`	50	simulation-cache commit interval, in batches
`TRAINING_BATCH_RETRY_ATTEMPTS`	3, delays 15, 60, 180 s	retries of a failed batch before the run stops
The χ block		
`CHI_N_FREQS`, `CHI_FREQ_BOUNDS`	6, (0.03, 0.3)	probes per observation in a simulated sweep; log-spaced band as multiples of the measured peak Ω_0
`CHI_F0`	0.15	ND drive amplitude of every probe (dimensional: 0.15 × the force scale)
`CHI_MIN_CYCLES`	2	a probe with fewer drive cycles in its segment is masked
`CHI_MAX_CYCLES`	20	ceiling on the cycles a lock-in runs over (a longer recording is truncated to its first 20)
`CHI_UHAT_MAX`	1.25	band-normalised frequency beyond which a probe is masked (training) or refused (a recording)
`CHI_K_PAD`, `CHI_K_MAX`	12, 24	slots per row (frozen into every posterior); largest pad the window accepts
`CHI_K_MIN_TRAIN`	2	floor of the per-batch probe-count draw in training
`CHI_ELEM_W`	6	channels per slot: u, $\log\lvert\chi\rvert$, cosine, sine, log cycles, mask
`CHI_PHI_DIM`, `CHI_RHO_HIDDEN`, `CHI_SET_OUT`	64, 128, 64	per-slot network width; set-network hidden width; output width (the 72 inputs become 64)
`CHI_BIN_DIM`	16	width of each knot's pooled vector (3 knots × 16 = 48)
`CHI_KNOTS`, `CHI_KNOT_SIGMA`, `CHI_KNOT_SHRINK`	$(-1, 0, 1)$, 0.6, 0.5	fixed band-normalised pooling knots; their Gaussian kernel width; shrinkage of an under-covered knot toward zero
Conditioning and the coordinate chain		
`WINSOR_PCT`	(0.001, 0.999)	winsorisation percentiles of the feature columns
`RANK_GAUSS_KNOTS`	1024	knots of the rank-Gaussian map
`REPARAM_ROTATE`	on	use the Fisher rotation

continued on the next page

Table 12, continued

Setting	Default	What it controls
`REPARAM_FISHER_M`, `REPARAM_FISHER_DZ`	48, 0.1	noise realisations per operating point; finite-difference step in z
`REPARAM_FISHER_POINTS`	8	operating points averaged for the rotation
`REPARAM_LOG_PARAMS`	empty	parameters inferred on a log scale
Flow and training		
`NSF_HIDDEN_FEATURES`, `NSF_NUM_TRANSFORMS`, `NSF_NUM_BINS`	128, 8, 10	spline-flow width, depth and bins per transform (the planned retrain uses 256 and 10 transforms)
`TRAINING_BATCH_SIZE`	512	optimiser mini-batch
`TRAINING_LEARNING_RATE`	10^{-3}	Adam learning rate
`TRAINING_STOP_AFTER_EPOCHS`	20	early-stopping patience, in epochs
`TRAINING_MAX_NUM_EPOCHS`	$2^{31}-1$	epoch ceiling handed to `sbi` (effectively none)
`TRAINING_NUM_ROUNDS`	1	rounds of training (single-round amortised inference)
Calibration and checks		
`SBC_N_CAL`	2000	calibration datasets (`-n-cal`)
`CAL_N_SCALES`	200	(t_{scale}, T) pairs the datasets are spread over (one batch each)
`CAL_RUN_SIZE`, `CAL_RUN_SIZE_MAX`	10, 256	calibration batch floor and ceiling
`TARP_NUM_BINS`	30	TARP coverage bins
`-posterior-samples`	1000	posterior draws per calibration dataset
`-repeats` (`sbc`)	10	independent repeats of the calibration
Inference and narrowing		
`-n-samples` (`infer`)	1000	posterior draws
`PPC_BIN_SIZE`	50	rows per simulation bin in the predictive check
`-directions` (`tsnpe`)	5	Fisher directions truncated per round
`-level` (`tsnpe`)	0.999	level of the highest-posterior interval along a direction
`-num-runs` (`tsnpe`)	5000	batches simulated in the round

B Where things live

Table 13: File map of the inference code.

Path	Role
`Resources/`	hand-edited inputs: `Bounds/` (which parameters are inferred, and so the mode), `Cells/`, `Units/`, `Models/`
`Artifacts/`	everything generated, one folder per record (not tracked by git)
`core/config.py`, `core/sim_config.py`	every default constant; the simulation configuration and the observation mode
`core/orchestrator.py`	the stage functions: `build_prior`, `build_posterior`, `build_embedding_net`, `validate_calibration`, `infer_and_visualize`, `tsnpe_round`, `generate_observations`
`core/runs.py`, `core/refusals.py`, `core/rng.py`	the public-entry wrapper and the run log; the pre-spend refusals; the seeds (including the calibration seed)
`core/forcing.py`	the drive (probe) waveforms
`core/Models/nadrowski_model.py`	the stochastic differential equations of the hair bundle
`core/Solvers/sdeint.py`, `core/Simulator/`	the Euler–Maruyama integrator with CUDA graphs; the simulator that wraps it
`core/SBI/Priors/`, `core/SBI/prior_screen.py`	the stability-screened prior and its construction
`core/SBI/pipeline.py`	the batch schedule and the generation of training batches
`core/SBI/statistics.py`, `core/SBI/summaries.py`	the 41 summary statistics and the validity flags
`core/SBI/chi.py`, `core/SBI/chi_probes.py`, `core/SBI/chi_encoder.py`	the lock-in estimate and its rows; probe placement; the set encoder of the χ block
`core/SBI/embedded_network.py`, `core/SBI/train.py`	the embedding network; training of the flow (built on `sbi`)
`core/SBI/decorrelate.py`, `core/SBI/reparam.py`, `core/SBI/derived.py`	the Fisher rotation; the coordinate chain and the posterior wrapper; the tier-1 derivation
`core/SBI/training_checkpoint.py`	the simulation cache: batch-by-batch commits and resume of a training-data generation
`core/SBI/truncate.py`	the truncation regions of a narrowing round

Table 13, continued

Path	Role
`core/SBI/analysis.py`, `core/SBI/ppc.py`, `core/SBI/overlay.py`	calibration data, SBC and TARP, informativeness; the predictive check; the overlay of a best fit on a recording
`core/SBI/observations.py`	building an observation from a recording
`core/artifacts/`	the artifact store: manifests, identity digests, provenance, loaders and their checks
`core/diagnostics/`	`sbc`, `identifiability`, `ablation` and `probes`
`core/tool/`	the command-line tool (`python -m core`)
`core/gui/`	the window (PySide6)
`tests/`	the pytest suites (fast gate: `pytest -m "not slow"`)
`docs/guide/`	the reader pages: the window, the command line, bringing recordings, the science, the architecture, the rules and traps, testing, the retrain runbook

References

[1] G. Papamakarios and I. Murray, Fast ϵ-free inference of simulation models with Bayesian conditional density estimation. In *Advances in Neural Information Processing Systems 29* (NeurIPS 2016).

[2] D. Greenberg, M. Nonnenmacher and J. Macke, Automatic posterior transformation for likelihood-free inference. In *Proceedings of the 36th International Conference on Machine Learning* (ICML 2019), PMLR 97:2404–2414.

[3] C. Durkan, A. Bekasov, I. Murray and G. Papamakarios, Neural spline flows. In *Advances in Neural Information Processing Systems 32* (NeurIPS 2019).

[4] A. Tejero-Cantero, J. Boelts, M. Deistler, J.-M. Lueckmann, C. Durkan, P. J. Gonçalves, D. S. Greenberg and J. H. Macke, sbi: A toolkit for simulation-based inference. *Journal of Open Source Software* 5(52):2505, 2020.

[5] S. Talts, M. Betancourt, D. Simpson, A. Vehtari and A. Gelman, Validating Bayesian inference algorithms with simulation-based calibration. arXiv:1804.06788, 2018.

[6] P. Lemos, A. Coogan, Y. Hezaveh and L. Perreault-Levasseur, Sampling-based accuracy testing of posterior estimators for general inference. In *Proceedings of the 40th International Conference on Machine Learning* (ICML 2023), PMLR 202:19256–19273.

[7] M. Deistler, P. J. Gonçalves and J. H. Macke, Truncated proposals for scalable and hassle-free simulation-based inference. In *Advances in Neural Information Processing Systems 35* (NeurIPS 2022).

[8] R. J. G. B. Campello, D. Moulavi and J. Sander, Density-based clustering based on hierarchical density estimates. In *Advances in Knowledge Discovery and Data Mining* (PAKDD 2013), LNCS 7819, pp. 160–172.

[9] P. E. Kloeden and E. Platen, *Numerical Solution of Stochastic Differential Equations*. Springer, Berlin, 1992.